A CASE-STUDY BASED REPORT

ON

Malicious Network Trafficing System Using K-Means and DBSCAN

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

J. Sai Shashank	**roll 2520030433**
K. Revanth	**roll 2520030545**
B. Chenna Malleswar	**roll 2520030456**

Under the Esteemed guidance of

G.Ravindra babu

Asst.Professor, Dept. of CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No.: 7815926816, www.kluniversity.in

INDEX

Abstract

1. Introduction
   1.1 Background
   1.2 Motivation for the Project
   1.3 Overview of the Work Carried Out
   1.4 Scope and Responsible-Use Statement
2. Problem Statement
   2.1 Case-Study Scenario
   2.2 Central Case-Study Question
   2.3 Formal Problem Definition
   2.4 Objectives that Guide the Study
   2.5 Assumptions and Limitations
3. Dataset and Exploratory Data Analysis
   3.1 About the Dataset
   3.2 Purpose of EDA
   3.3 Preparation Steps
   3.4 EDA Exploration - Implementation
   3.5 Summary of EDA Findings
4. Algorithm Selection and Implementation
   4.1 K-Means Clustering
   4.2 DBSCAN Clustering
   4.3 Evaluation and Comparison
5. Conclusion

Abstract

Computer networks generate large volumes of traffic, and a small portion of this traffic may be malicious or unusual. Detecting suspicious behaviour is important because attacks can disrupt services, expose information, or consume network resources. This case study explores an unsupervised machine-learning approach for grouping network traffic records using K-Means and DBSCAN. The project uses the CSV file “Malicious_Network_Traffic_500.csv” as its input dataset. The workflow loads and inspects the records, checks missing values, selects numerical traffic features, scales feature values, and applies both clustering algorithms. K-Means groups records around a specified number of centroids, while DBSCAN forms dense groups and can mark isolated points as noise. Principal Component Analysis (PCA) is used to support visual exploration in a lower-dimensional space. Cluster labels are compared with any available ground-truth labels only for interpretation; clustering itself does not require labels. The study discusses how the methods can help reveal traffic patterns and potentially unusual records, while noting that a cluster or noise point is not automatically proof of an attack. Final cluster counts and evaluation scores should be taken from the actual program output rather than assumed in advance.

Keywords: Malicious Network Traffic, K-Means, DBSCAN, Clustering, Anomaly Detection, PCA, Machine Learning.

1. Introduction

1.1 Background

Modern organisations depend on computer networks for communication, learning, banking, cloud services, and business operations. Network traffic consists of records describing connections or flows, such as flow duration and packet counts. Most traffic is legitimate, but some flows may show unusual patterns associated with scanning, flooding, suspicious connections, or other malicious activity. Manually inspecting every record is difficult when the volume of traffic is large.

Machine learning can help analysts explore traffic patterns. Supervised models learn from labelled examples, whereas unsupervised clustering groups records by similarity without requiring a known label for every record. This project studies two clustering methods: K-Means and DBSCAN. K-Means seeks compact groups around centroids. DBSCAN identifies groups based on density and can label sparse points as noise.

1.2 Motivation for the Project

The motivation is to explore whether clustering can reveal meaningful groups in network-flow data and help flag records that differ from common patterns. Using two algorithms provides a useful comparison: K-Means is simple and efficient when the expected number of groups is known, while DBSCAN can discover irregularly shaped dense groups and does not require a preselected cluster count. Their different assumptions make them complementary tools for exploratory security analysis.

1.3 Overview of the Work Carried Out

Loaded the CSV dataset with pandas and inspected its dimensions, column names, data types, sample records, and missing values.

- Loaded the CSV dataset with pandas and inspected its dimensions, column names, data types, sample records, and missing values.
- Selected suitable numerical network-traffic features, such as flow duration and forward/backward packet counts where available.
- Prepared the feature matrix and applied StandardScaler so that variables with larger numeric ranges would not dominate distance calculations.
- Applied K-Means and DBSCAN to the prepared features.
- Used PCA to project the feature data into two dimensions for visual inspection of cluster patterns.
- Reviewed cluster sizes, noise assignments, and available evaluation measures; interpreted the results cautiously.

1.4 Scope and Responsible-Use Statement

This project analyses previously collected network-traffic records. It is an exploratory clustering study and is not presented as a production intrusion-prevention system. A cluster assignment or DBSCAN noise label must not be treated as a definitive security verdict. The work should be used for defensive analysis on networks or datasets that the user is authorised to inspect. Human review and additional evidence are required before responding to a suspected threat.

2. Problem Statement

2.1 Case-Study Scenario

Consider an organisation that stores thousands of network-flow records for monitoring and investigation. Analysts want to understand which records share similar traffic behaviour and which records are sufficiently different to deserve closer inspection. The dataset may not contain reliable labels for every record, so a method that can explore the data without depending on labelled examples is useful.

2.2 Central Case-Study Question

How effectively can K-Means and DBSCAN group records in the malicious network-traffic dataset, and how useful are their cluster and noise assignments for identifying traffic patterns that warrant further investigation?

2.3 Formal Problem Definition

Let each network record be represented by a feature vector containing numerical traffic attributes. The task is to group similar vectors based on their distances or local density. K-Means partitions the data into K clusters by minimising within-cluster squared distances. DBSCAN groups points that have enough neighbours within a chosen radius (eps) and treats points that do not belong to dense regions as noise. The output is a cluster identifier for each record, with DBSCAN potentially assigning a noise label of -1.

2.4 Objectives that Guide the Study

• Understand the structure and quality of the selected network-traffic dataset.

• Prepare numerical features for distance-based clustering.

• Apply K-Means and DBSCAN to the same prepared data.

• Visualise cluster structure with PCA for exploratory interpretation.

• Compare cluster size, noise points, and suitable internal metrics where applicable.

• Discuss limitations and explain why clustering output requires security analyst validation.

2.5 Assumptions and Limitations

Assumptions

• The CSV records and their feature values are sufficiently representative for an initial experiment.

• The selected numeric columns describe relevant aspects of network-flow behaviour.

• Feature scaling is appropriate because both algorithms depend on distance or density.

Limitations

• The dataset is small compared with real operational network traffic, so findings may not generalise.

• K-Means requires the number of clusters to be chosen and is sensitive to outliers and initialisation.

• DBSCAN is sensitive to eps and min_samples; one parameter setting may not fit clusters of different densities.

• High-dimensional distance calculations can be difficult to interpret, and PCA visualisation may hide some structure.

• An anomaly is not necessarily malicious, and a malicious record may appear inside a large normal-looking cluster.

3. Dataset and Exploratory Data Analysis

3.1 About the Dataset

The project input file is named “Malicious_Network_Traffic_500.csv”. The name indicates a small traffic dataset, but its exact number of rows, columns, labels, and missing values must be confirmed by running the project code. The workflow inspects the dataset using pandas, including its shape, first records, data types, and missing-value summary. Features in the current project context include Flow Duration, Total Fwd Packets, and Total Backward Packets; the complete feature list should be taken from the CSV header.

The clustering process should use explanatory traffic features only. If the file contains a target or attack-label column, it should be excluded from the input features used to fit the unsupervised algorithms. Labels can be retained separately for post-hoc interpretation, provided they are not used to form the clusters.

3.2 Purpose of EDA

Exploratory Data Analysis (EDA) helps determine whether the records are suitable for clustering. It is used to identify missing or invalid values, understand feature ranges, detect constant or highly skewed columns, and review the distribution of any available labels. Since distance-based algorithms are affected by feature scale, EDA also helps identify the need for scaling and the possibility that extreme values may influence cluster formation.

3.3 Preparation Steps

- Read the CSV file into a pandas DataFrame and inspect data.shape, data.head(), data.info(), and data.isnull().sum().
- Identify numeric traffic features and exclude identifiers or label columns from the clustering input.
- Handle missing or infinite values using a documented strategy; do not silently replace values without checking their meaning.
- Scale selected numerical features using StandardScaler before calculating distances.
- Fit K-Means and DBSCAN on the prepared feature matrix.
- Use PCA to create a two-dimensional projection for plotting; use the original scaled feature matrix for clustering unless the implementation explicitly states otherwise.

3.4 EDA Exploration - Implementation

A typical inspection block for the project is shown below. The printed values should be copied from the actual execution when documenting the final dataset statistics.

```
import pandas as pd\n\ndata = pd.read_csv("Malicious_Network_Traffic_500.csv")\nprint("Shape:", data.shape)
\nprint(data.head())\nprint(data.info())\nprint("Missing values:", data.isnull().sum().sum())
```

After inspection, numerical features are scaled because features such as packet counts and flow duration can have very different numeric ranges. Without scaling, a feature with large values may dominate Euclidean distance and distort the clusters.

3.5 Summary of EDA Findings

- The dataset file used by the project is Malicious_Network_Traffic_500.csv.
- The exact dataset dimensions and missing-value count should be reported from the program output.
- Traffic-flow variables may have different ranges, making feature scaling an important preparation step.
- Any label column should be kept separate from the unsupervised clustering features.
- PCA plots are useful for visual inspection but do not alone prove that clusters are meaningful or accurate.

4. Algorithm Selection and Implementation

4.1 K-Means Clustering

K-Means is a partition-based clustering algorithm. It starts with K centroids, assigns each record to the nearest centroid, recalculates each centroid as the mean of the records assigned to it, and repeats until the assignments stabilise or the iteration limit is reached. The algorithm is straightforward and often efficient for numeric datasets, but the value of K must be selected before fitting.

In this project, K-Means is applied to the scaled traffic-feature matrix. The chosen K value should be justified using a method such as the elbow curve and, where appropriate, the silhouette score. Results can be examined by counting records in each cluster and plotting a PCA projection coloured by cluster identifier.

4.2 DBSCAN Clustering

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise. It uses two main parameters: eps, the neighbourhood radius, and min_samples, the minimum number of nearby points needed to form a dense region. It expands clusters from core points and can mark points that do not belong to a dense region as noise, conventionally using label -1.

DBSCAN is useful because it does not require the number of clusters in advance and can identify noise points. However, its results can change substantially when eps and min_samples change. Standardising features is especially important because the neighbourhood radius is distance-based. A DBSCAN noise point should be treated as a candidate for review, not automatic proof of malicious activity.

4.3 Implementation Workflow

- Load and inspect the dataset.
- Select and clean the traffic features used for clustering.
- Scale the feature matrix using StandardScaler.
- Fit K-Means with a documented K value and random_state for reproducibility.
- Fit DBSCAN with documented eps and min_samples values.
- Count the records assigned to each cluster and the number assigned to DBSCAN noise.
- Use PCA to visualise the cluster assignments and inspect overlap or separation.
- If ground-truth labels exist, compare them only after clustering, using them for interpretation rather than model fitting.

4.4 Example Code Structure

```
from sklearn.preprocessing import StandardScaler\nfrom sklearn.cluster import KMeans, DBSCAN\nfrom sklearn.decomposition import PCA\nfrom sklearn.metrics import silhouette_score\n\n# X should contain only selected numeric traffic features\nX_scaled = StandardScaler().fit_transform(X)\n\nkmeans = KMeans(n_clusters=3, random_state=42, n_init=10)\nkmeans_labels = kmeans.fit_predict(X_scaled)\n\ndbscan = DBSCAN(eps=0.5, min_samples=5)\ndbscan_labels = dbscan.fit_predict(X_scaled)\n\nprojection = PCA(n_components=2).fit_transform(X_scaled)
```

The values shown in this example are starting settings, not guaranteed optimal parameters. The final report should state the actual parameter values used by the submitted program. If all records receive one cluster or the silhouette score cannot be calculated, report that outcome rather than forcing a score.

4.5 Evaluation and Comparison

K-Means and DBSCAN should be compared using the same selected and scaled input features. Useful observations include the number and sizes of clusters, whether DBSCAN identifies noise, and whether the PCA plot shows visible separation. The silhouette score may be used when there are at least two meaningful clusters and the metric is valid for the assignments. For DBSCAN, noise

points require careful handling when calculating internal metrics. If trusted labels are available, external measures can be added as a separate analysis.

K-Means is generally preferable when compact groups and a chosen cluster count are reasonable assumptions. DBSCAN is more useful when dense regions and outliers are of interest, but it can struggle when cluster densities vary. Neither method by itself confirms that a record is malicious.

4.6 Results Reporting Template

Record the actual output from the run in the following format. Do not fill these fields with estimated or invented values.

Item	K-Means	DBSCAN
Parameters	K = [actual value]	eps = [actual value], min_samples = [actual value]
Number of clusters	[actual count]	[actual count, excluding noise]
Noise records	Not explicitly labelled as noise	[actual count with label -1]
Silhouette score	[actual score, if valid]	[actual score, if valid]

5. Conclusion

This case study presents an unsupervised approach to exploring malicious network traffic using K-Means and DBSCAN. The workflow focuses on dataset inspection, selection of traffic features, scaling, clustering, and PCA-based visualisation. The algorithms provide different views of the same data: K-Means assigns every record to one of a fixed number of clusters, while DBSCAN groups dense regions and can identify sparse records as noise.

The most appropriate method depends on the structure of the data and the selected parameters. Cluster counts, noise counts, and evaluation scores must be taken from the actual execution of the project and should not be assumed in advance. In future work, the system can be improved by testing multiple parameter settings, using larger and more varied traffic datasets, comparing results with trusted labels, and validating suspected anomalies with additional network-security evidence.

Overall, clustering can support traffic exploration and help prioritise records for investigation, but it should be used as a decision-support technique rather than a standalone malicious-traffic verdict.

Signature of the Team Members

Roll Numbers: 2520030433, 2520030545, 2520030546